Automated Timetable Scheduling

User Manual

Department of Computer Science and Engineering
Indian Institute of Information Technology, Dharwad

Version: 1.1
Date: November 27, 2025

Prepared by: Semi Colon

For internal academic use

Contents

1 Overview **2**

2 Installation **4**

3 Usage Scenarios with Screen Shots **6**

4 Sample Inputs, Config Files **11**

5 FAQ's **14**

6 Any other section relevant to your software usage **16**

1 Overview

The **Automated Timetable Scheduling** software is a Python-based system that generates academic timetables and, optionally, exam-related schedules such as exam timetables and seating allocations. It is designed to support multiple batches, faculty members, courses, and rooms while respecting capacity limits and various institutional constraints.

Purpose

The purpose of this software is to:

- Reduce manual effort and human error in timetable creation.
- Provide a repeatable, configurable scheduling process.
- Generate Excel-based timetables that are easy to share and edit.
- Support extensions for exam timetables and exam seating plans.

Key Features

- Automatic generation of weekly timetables for multiple batches.
- Faculty-wise timetable generation.
- Basic clash checking:
  - A faculty member is not assigned to more than one class in the same slot.
  - A batch does not have two different courses in the same slot.
  - A room is not double-booked in the same slot.
- Support for room capacity and room type (lecture / lab).
- Use of structured Excel inputs: `faculty.xlsx`, `courses.xlsx`, `rooms.xlsx`, `batches.xlsx`.
- Export of results in `.xlsx` format for further formatting or printing.

Scope

The current version focuses mainly on:

- Regular class timetable scheduling for one semester.
- Basic exam timetable and seating (if enabled/configured).
- Constraint checking based on:
  - Faculty availability.

  - Room capacity and type.
  - Weekly hours per course.

Advanced optimisation (like perfectly balanced load for faculty or minimizing gaps) is outside the scope of the current version but can be added later.

Intended Users

- **Timetable coordinators** and **academic administrators** who prepare official institute timetables.
- **Faculty in-charges** who collect and verify input data.
- **Developers and student teams** who wish to extend the timetable logic.

High-Level Workflow

At a high level, the system works as follows:

1. Collect input data in Excel templates (faculty, courses, rooms, batches).
2. Run the scheduling script.
3. The scheduler attempts to allocate each course to suitable slots and rooms.
4. The software writes the results to Excel files (batch-wise and faculty-wise).
5. Any unscheduled courses are listed with reasons so they can be fixed manually.

2 Installation

This section explains how to set up the software on a typical system. The examples assume a command line environment.

System Requirements

- **Operating System:** Windows 10/11, Linux, or macOS.
- **Python:** Version 3.8 or above.
- **RAM:** Minimum 4 GB recommended.
- **Disk Space:** At least 200 MB free for code, inputs, and outputs.
- **Software:** An application that can open `.xlsx` files (e.g., Microsoft Excel, LibreOffice).

Step 1: Obtain the Source Code

You can place the project folder anywhere on your system. Example layout:

```
AutomatedTimetable/
    timetable_automation/
        main.py
        scheduler.py
        config.py
        ...
    data/
    outputs/
    tests/
    requirements.txt
```

Step 2: Create a Virtual Environment (Recommended)

Windows:

```
python -m venv venv
venv\Scripts\activate
```

Linux / macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Step 3: Install Dependencies

If a `requirements.txt` file is present, run:

```
pip install -r requirements.txt
```

If not, install the essential packages manually:

```
pip install pandas openpyxl numpy pytest
```

Step 4: Verify Installation

Run a quick import test:

```
python -c "import pandas, openpyxl, numpy; print('OK')"
```

If you see `OK`, the basic dependencies are set up correctly.

Common Installation Issues

Issue: `pip` Not Found Add Python to the PATH during installation or use the full Python path, such as:

```
C:\Python311\python.exe -m pip install pandas
```

Issue: Permission Denied On Linux/macOS, you may need:

```
pip install --user pandas
```

or to use a virtual environment as shown above.

Issue: Different Python Versions Check version with:

```
python --version
```

and use `python3` if needed.

3 Usage Scenarios with Screen Shots

This section describes typical ways in which the software is used and shows screenshots of important steps and outputs.

Scenario 1: Generating the Class Timetable

Step-by-Step

1. Prepare all four input files in the `data/` directory: `faculty.xlsx`, `courses.xlsx`, `rooms.xlsx`, `batches.xlsx`.
2. Adjust any necessary values in `config.py` (days of week, slots per day, etc.).
3. Open a terminal or command prompt in the project root folder.
4. Run:

   ```
   python timetable_automation/main.py
   ```

5. Watch the console output for progress messages and errors.
6. Open the generated Excel files in the `outputs/` (or configured) folder.

Console Output Screenshot

```
Generating student timetable for CSE-3-A...
Saved timetable sheet: First_Half
Saved timetable sheet: Second_Half
Formatted student timetable saved in CSE-3-A_timetable.xlsx

Generating student timetable for CSE-3-B...
Saved timetable sheet: First_Half
Saved timetable sheet: Second_Half
Formatted student timetable saved in CSE-3-B_timetable.xlsx

Generating student timetable for CSE-1-A...
Saved timetable sheet: First_Half
Saved timetable sheet: Second_Half
Formatted student timetable saved in CSE-1-A_timetable.xlsx

Generating student timetable for CSE-1-B...
Saved timetable sheet: First_Half
Saved timetable sheet: Second_Half
Formatted student timetable saved in CSE-1-B_timetable.xlsx

Generating student timetable for CSE-5-A...
Saved timetable sheet: First_Half
Saved timetable sheet: Second_Half
Formatted student timetable saved in CSE-5-A_timetable.xlsx

Generating student timetable for CSE-5-B...
Saved timetable sheet: First_Half
Saved timetable sheet: Second_Half
Formatted student timetable saved in CSE-5-B_timetable.xlsx
```

The console output typically shows messages for:

- Reading and validating input sheets.
- Scheduling progress (e.g., assigning courses to slots).
- Summary of scheduled and unscheduled courses.
- Paths of saved timetable files.

Scenario 2: Viewing a Generated Timetable

After a successful run, you can open a batch timetable like `CSE-3-A_timetable.xlsx`.

	07:30-09:00	09:00-10:00	10:00-10:30	10:30-10:45	10:45-11:00	11:00-12:00	12:00-12:15	12:15-12:30	12:30-13:15	13:15-14:00	14:00-14:30	14:30-15:30	15:30-15:40	15:40-16:00	16:00-16:30	16:30-17:10	17:10-17:30	17:30-17:40	17:40-18:30
Monday		CS264 (C101)		BREAK	CS261 (C101)			BREAK			MA261 (C101)		BREAK	CS262 (C101)			BREAK		
Tuesday		CS264 (C101)		BREAK	CS261 (C101)			BREAK			MA261 (C101)		BREAK	CS262 (C101)			BREAK		
Wednesday		CS264T (C101)	BREAK		CS261T (C101)		BREAK	MA261T (C101)			CS263 (Lab-L107)				BREAK	Elective_1T		BREAK	
Thursday		CS263 (C101)		BREAK	Elective_1			BREAK											
Friday		CS262 (Lab-L107)				BREAK					CS263 (C101)		BREAK	Elective_1			BREAK		

S.No	Course Code	Course Title	L-T-P-S-C	Faculty	Color
1	CS264	Computer Networks	3-1-0-0-4	Dr. Prabhu Prasad BM	
2	CS261	Operating Systems	3-1-0-4-2	Dr. Suvadip Hazra	
3	MA261	Differential Equations	3-1-0-0-2	Dr. Anand Barangi	
4	CS262	Software design tools and technique	3-0-2-0-3	Dr. Sunil Kumar PV	
5	CS263	Design & Analysis of Algorithms	3-0-2-0-4	Dr. Malay	

Typically:

- Rows correspond to days (e.g., Monday, Tuesday, etc.).
- Columns correspond to time slots during the day.
- Each cell usually contains: `CourseCode -- Room -- FacultyID`.

Scenario 3: Verifying Logic Using Tests

The project may include automated tests (e.g., using `pytest`) to validate course parsing, slot logic, and constraints.

tests > test_course.py > ...

```python
import pytest
from timetable_automation import Course

def test_course_valid_fields():
    row = {
        "Course_Code": " CS101 ",
        "Course_Title": " Intro to Programming ",
        "Faculty": "Prof X",
        "L-T-P-S-C": "3-1-0-0-4",
        "Semester_Half": "1",
        "Elective": "0",
    }
    c = Course(row)
    assert c.code == "CS101"
    assert c.title == "Intro to Programming"
    assert c.faculty == "Prof X"
    assert (c.L, c.T, c.P) == (3, 1, 0)
    assert not c.is_elective

def test_course_malformed_ltp():
    row = {
        "Course_Code": "CS102",
        "Course_Title": "Error Test",
        "Faculty": "",
        "L-T-P-S-C": "3-1-X-0-4",   # malformed
    }
    c = Course(row)
    assert (c.L, c.T, c.P) == (0, 0, 0)

def test_course_missing_optional_fields():
    row = {
        "Course_Code": "CS103",
        "L-T-P-S-C": "3-0-0-0-3"
    }
    c = Course(row)
    assert c.faculty == ""
    assert c.sem_half == "0"

def test_course_elective_detection():
    row = {
        "Course_Code": "CS104",
        "L-T-P-S-C": "3-1-0-0-4",
        "Elective": "1",
    }
    c = Course(row)
    assert c.is_elective
```

To run tests:

```
pytest -q
```

If a test fails, read the error output to understand which part of the logic or data needs correction.

Scenario 4: Regenerating Timetables After Changes

1. Modify one or more input Excel files (for example, change a faculty member's available slots).
2. Save the Excel files.
3. Rerun the command:

   ```
   python timetable_automation/main.py
   ```

4. Compare the new timetables against the older ones to confirm that the changes are reflected.

4 Sample Inputs, Config Files

This section provides sample content for input files and basic configuration. You can use these as reference templates when preparing your data.

Sample Faculty Input (`faculty.xlsx`)

Typical columns:

Column	Example	Description
FacultyID	F101	Unique faculty identifier
Name	Dr. A. Kumar	Full name of the faculty member
Department	CSE	Department or branch
AvailableSlots	MON_1;MON_2;TUE_1	Semicolon-separated slot tokens
MaxHoursPerDay	4	Maximum teaching hours per day (optional)

Example Row (CSV format)

```
FacultyID,Name,Department,AvailableSlots,MaxHoursPerDay
F101,Dr. A. Kumar,CSE,MON_1;MON_2;TUE_1;WED_3,4
```

Sample Courses Input (`courses.xlsx`)

Column	Example	Description
CourseCode	CSE203	Unique course code
Title	Data Structures	Course title
Type	Lecture	Lecture / Lab / Tutorial
FacultyID	F101	Faculty teaching the course
Batch	CSE-3-A;CSE-3-B	One or more batches
HoursPerWeek	4	Total hours per week
PreferredRoomType	Lecture	Optional preference
Elective	0 / 1	Marks elective courses

Example Row (CSV format)

```
CourseCode,Title,Type,FacultyID,Batch,HoursPerWeek,PreferredRoomType,Elective
CSE203,Data Structures,Lecture,F101,CSE-3-A,4,Lecture,0
```

Sample Rooms Input (`rooms.xlsx`)

Column	Example	Description
RoomID	C101	Room identifier
Capacity	60	Maximum seating capacity
RoomType	Lecture	Lecture / Lab / Tutorial room
AvailableSlots	MON_1;MON_2;...	Optional restricted slots

Example Row (CSV format)

```
RoomID,Capacity,RoomType,AvailableSlots
C101,60,Lecture,MON_1;MON_2;TUE_1;TUE_2;WED_3
```

Sample Batches Input (`batches.xlsx`)

Column	Example	Description
BatchID	CSE-3-A	Unique batch code
Program	BTech	Name of the programme
Semester	3	Semester number
StudentsCount	60	Number of students (optional)
CourseCodes	CSE203;MAT101	Courses for this batch

Example Row (CSV format)

```
BatchID,Program,Semester,StudentsCount,CourseCodes
CSE-3-A,BTech,3,60,CSE203;MAT101;PHY105
```

Sample Configuration Snippet (`config.py`)

```
DAYS = ['MON', 'TUE', 'WED', 'THU', 'FRI']
SLOTS_PER_DAY = 6

SLOT_TIMINGS = {
    1: '09:00-10:00',
    2: '10:00-11:00',
    3: '11:15-12:15',
    4: '13:15-14:15',
    5: '14:15-15:15',
    6: '15:30-16:30',
```

```
}

OUTPUT_DIR = 'outputs/'
```

5 FAQ's

This section answers frequently asked questions related to installation, usage, and troubleshooting.

Q1. The program does not start when I run the command. What should I check?

Answer:

- Ensure you are running the command from the correct project root directory.
- Confirm that Python is installed and added to your PATH.
- Check you are using the correct command:

  ```
  python timetable_automation/main.py
  ```

- If you use a virtual environment, make sure that it is activated.

Q2. I get an error about missing modules like `pandas` or `openpyxl`.

Answer:

- Install dependencies using:

  ```
  pip install -r requirements.txt
  ```

- Or manually install:

  ```
  pip install pandas openpyxl numpy
  ```

Q3. Why are some courses not appearing in the timetable?

Answer:

- Check the `unscheduled_courses.xlsx` file (if generated).
- Typical reasons:
  - No available slot for that faculty.
  - No room that satisfies capacity or type constraints.
  - Weekly hours for the course are too high for the available timetable.
- Try increasing available slots, adding more rooms, or adjusting constraints.

Q4. How can I change the days or number of slots per day?

Answer: Edit the settings in `config.py`, such as:

```
DAYS = ['MON', 'TUE', 'WED', 'THU', 'FRI', 'SAT']
SLOTS_PER_DAY = 7
```

Ensure that your input data (AvailableSlots in Excel) matches these tokens.

Q5. Can I reuse the same setup for the next semester?

Answer: Yes. You simply update the Excel input files with the new courses, batches, and faculty assignments. The configuration and code can remain the same unless the structure of the timetable changes.

Q6. Can this system generate exam timetables and seating plans?

Answer: If the exam-related extensions are present in your codebase:

- Prepare exam-specific input sheets (for example, `exam_courses.xlsx`).
- Run the script in exam mode (e.g., with a flag like `--exam`, if implemented).
- Seating plans can then be generated based on room capacity and student counts.

Q7. How do I export timetables to PDF?

Answer:

- Open the Excel timetable file.
- Use "Save As" or "Export" to PDF from your spreadsheet application.
- You can adjust page breaks and orientation (landscape) before exporting.

6 Any other section relevant to your software usage

This section collects additional information that is useful for long-term usage, maintenance, and extension of the software.

Best Practices for Data Preparation

- **Consistent IDs:** Use consistent `FacultyID`, `RoomID`, and `BatchID` across all sheets.
- **No extra spaces:** Avoid trailing spaces in Excel cells, especially in IDs and column headers.
- **Realistic hours:** Ensure that `HoursPerWeek` values are realistic for the number of available slots.
- **Validate early:** Run small test timetables before using large datasets.
- **Backup files:** Keep backups of all input Excel files before running major changes.

Troubleshooting Guide

Problem: Scheduler crashes with a KeyError.

- Verify that the column names in the Excel sheets match exactly what the code expects.
- Check for typos like "RoomID " (with space) instead of "RoomID".

Problem: Many courses show up as unscheduled.

- Check faculty availability: maybe they have too few available slots.
- Check room constraints: there may not be enough rooms of the required type.
- Reduce weekly hours for some courses or add more slots/days.

Problem: A course appears in an unexpected room.

- Ensure that `PreferredRoomType` in `courses.xlsx` matches the rooms' types in `rooms.xlsx`.
- Check the scheduling rules in the code (room selection part) if you want to enforce different preferences.

Glossary of Terms

Batch A group of students (e.g., CSE-3-A).

Slot A defined time period in the timetable, such as "MON_1".

Capacity Maximum number of students a room can hold.

Elective A course that is optional or chosen from a list.

L-T-P Lecture–Tutorial–Practical structure of a course.

Extending the Software

Developers can extend the system by:

- Adding new constraints (e.g., no classes after a certain time for specific batches).
- Implementing advanced optimisation algorithms to reduce free gaps.
- Integrating with a web front-end for online timetable viewing.

General Steps to Add a New Constraint

1. Identify where in the scheduling logic a course is checked before assignment.
2. Add a function like `is_valid_assignment(course, slot, room)` that includes the new rule.
3. Update tests (if present) to cover the new behaviour.

Maintenance and Versioning

- Keep a simple change log text file or section documenting:
  - Date of change.
  - Description of change (e.g., "Added elective constraint", "Updated config for new semester").
- Use version numbers (e.g., 1.0, 1.1, 1.2) to track releases.
- Regularly test after code or configuration updates.

Example Command Summary

```
# Generate regular timetables
python timetable_automation/main.py

# (If available) generate exam timetables
python timetable_automation/main.py --exam
```

```
# Run automated tests
pytest -q
```